Homework 2 (Written): Semantics

17-355/17-665/17-819: Program Analysis
Fraser Brown, Yiliang Liang

Due: Tuesday, September 15, 2026 (11:59 PM) (120 points)

Assignment Objectives:

- Construct derivation trees of a program in WHILE using inference rules.
- Precisely specify language features using both small- and big-step semantic rules.
- Carefully consider the benefits of small- versus big-step rules for specifying language features.
- Practice and demonstrate the use of induction on the structure of derivations to prove conjectures about the semantic rules for WHILE.

Handin Instructions *(4 points)*. Please submit your assignment through GradeScope by the due date. When submitting, please indicate which pages of the PDF correspond to each homework question. Putting page breaks between questions makes this simpler. Typesetting is not required, but is suggested; you may submit photos of handwritten answers, but they must be clear and legible. Feel free to typeset your proofs using your favorite LaTeX package. If you do not have one, you may be interested in `mathpartir`, which you can find on the website (look at the schedule on the date where hw2 is due and download `mathpartir.zip`). To see how to write some inference rules, compile the example `mathpartir.tex` with, e.g., `pdflatex`. To use it for your assignment, include `mathpartir.sty` in your tex file (i.e., (`\usepackage {mathpartir}`). Alternatively, you can also modify `mathpartir.tex` directly.

If you find any typos or errors in the assignment, please let us know by posting on Piazza or sending an email to the instructors. We will post any corrections on Piazza.

Question 1, Derivations, *(20 points).* In this question, you will use the big-step operational semantics rules of WHILE to construct a derivation tree that proves the behavior of a program. Consider the following program S where the line splits and indentations are for readability only:

$a := 3;$
if $a + b > 5 + 3$ then
  skip
else (
  $d := 5 + a;$
  $a := d + e$
)

The initial environment E_{init} is as follows:

$$E_{\text{init}}(b) = 2 \quad \text{and}$$
$$E_{\text{init}}(x) = 0 \quad \text{for all other variables } x.$$

The goal of the proof is to show that $\langle E_{\text{init}}, S\rangle \Downarrow E_{\text{final}}$ for some appropriate environment E_{final}.

We have sketched out the derivation tree for the proof below. Your task is to fill in the missing details and complete the proof.

$$\dfrac{\dfrac{\mathcal{D}_1}{\langle E_{\text{init}}, a := 3\rangle \Downarrow E'}\ ?_2 \quad \dfrac{\dfrac{\mathcal{D}_2 \quad \mathcal{D}_3}{\langle E', a + b > 5 + 3\rangle \Downarrow_b \text{false}}\ ?_4 \quad \mathcal{D}_4}{\langle E', \text{if } a + b > 5 + 3 \text{ then skip else } (d := 5 + a; a := d + e)\rangle \Downarrow E_{\text{final}}}\ ?_3}{\langle E_{\text{init}}, S\rangle \Downarrow E_{\text{final}}}\ ?_1$$

In the above derivation tree, $\mathcal{D}_1$, $\mathcal{D}_2$, $\mathcal{D}_3$, and $\mathcal{D}_4$ represent the sub-derivations that you need to fill in in this question. The places labeled $?_1$, $?_2$, $?_3$, and $?_4$ are the names of the rules, which you will also fill in. In answering this question, you may assume that the big-step operational semantics rules for WHILE are as given in the lecture notes. You may also use a few additional rules for boolean expressions (not given in the lecture notes), which are provided at the end of this question.

Part (a): What is the final environment E_{final}? Give the answer in terms of the values of variables a, b, d, and e in the environment.

Part (b): What is the environment E' in the derivation tree? Give the answer in terms of the values of variables a, b, d, and e in the environment.

Part (c): What are the names of the rules $?_1$, $?_2$, $?_3$, and $?_4$ in the derivation tree?

Part (d): Please fully write out the sub-derivations $\mathcal{D}_1$, $\mathcal{D}_2$, $\mathcal{D}_3$, and $\mathcal{D}_4$. Feel free to create additional environments and sub-derivations; just make sure to label them clearly. Make sure to cite the appropriate rules and use the correct versions of the judgements ($\Downarrow$, $\Downarrow_a$, or $\Downarrow_b$). In the end, you should have a complete derivation tree where the leaves are all axioms (inference rules with empty premises).

Rules for Boolean Expressions:

$$\frac{}{\langle E, \text{true}\rangle \Downarrow_b \text{true}}\ \textit{big-true} \qquad \frac{}{\langle E, \text{false}\rangle \Downarrow_b \text{false}}\ \textit{big-false}$$

$$\frac{\langle E, a_1\rangle \Downarrow_a n_1 \quad \langle E, a_2\rangle \Downarrow_a n_2}{\langle E, a_1\ \text{op}_r\ a_2\rangle \Downarrow_b n_1\ \text{op}_r\ n_2}\ \textit{big-opr}\ \text{where } \text{op}_r \in \{>, <, =, \geq, \leq, \neq\}$$

$$\frac{\langle E, b_1\rangle \Downarrow_b b_1' \quad \langle E, b_2\rangle \Downarrow_b b_2'}{\langle E, b_1\ \text{op}_b\ b_2\rangle \Downarrow_b b_1'\ \text{op}_b\ b_2'}\ \textit{big-opb}\ \text{where } \text{op}_b \in \{\wedge, \vee\}.$$

Question 2, `let` Statement, *(20 points).* Consider the WHILE language (not WHILE3ADDR!) extended with a new statement "`let` x = e `in` s". The informal semantics of this construct is that the expression e is evaluated; a new local variable x is created with lexical scope c; and x is initialized with the result of evaluating e. Then the statement s is evaluated in c. For exposition/convenience, we also extend WHILE with statement "`print` e" which evaluates the e and "displays the result" in some un-modeled manner (it is otherwise similar to `skip`). Additionally, we assume that all environments E begin with a value of 0 for every variable that will be used in any program. We therefore expect the following code to display "3 2 1 5" (the curly braces are syntactic sugar):

$x := 1;$
$y := 2;$
let $x = 3$ in {
  print x;
  print y;
  $x := 4;$
  $y := 5;$
};
print x; print y

Part (a): Extend the big-step operational semantics judgment $\langle E, s\rangle \Downarrow E'$ with one new rule for dealing with the *let* statement. Pay careful attention to the scope of the newly declared variable and to changes to other variables.

Part (b): Extend the small-step operational semantics judgment $\langle E, s\rangle \rightarrow \langle E', s'\rangle$ to account for the *let* statement.

Question 3, Exceptional semantics, *(25 points).* One way to handle error situations (like divide-by-zero, mentioned in class) generally is to explicitly introduce error handling into the language. We thus add to WHILE integer-valued *exceptions* (or *run-time errors*), as in Java, ML or C#. We introduce a new sort T to represent command terminations, which can either be normal or exceptional (with an exception value $n \in \mathbb{Z}$):

$$\begin{array}{rcll} T & ::= & E & \text{"normal termination"} \\ & | & E\ \texttt{exc}\ n & \text{"exceptional termination"} \end{array}$$

We use t to range over T. We then redefine our big-step operational semantics judgment:

$$\langle E, S\rangle \Downarrow T$$

The interpretation of

$$\langle E, S\rangle \Downarrow E' \texttt{ exc } n$$

is that statement S terminated abruptly by throwing an exception with value $n \in \mathbb{Z}$ at a point in S's execution when the state was E'. We only model one type of exception, but every exception has an integer "argument" n (or "payload" or "value") that is set when the exception is thrown and available when the exception is caught.

Our previous statement rules must now be updated to account for exceptions, as in:

$$\frac{\langle E, S_1\rangle \Downarrow E' \texttt{ exc } n}{\langle E, S_1; S_2\rangle \Downarrow E' \texttt{ exc } n}\ \textsf{seq1} \qquad \frac{\langle E, S_1\rangle \Downarrow E' \quad \langle E', S_2\rangle \Downarrow t}{\langle E, S_1; S_2\rangle \Downarrow t}\ \textsf{seq2}$$

We also introduce two new statements:

- `throw` e: raise an exception with argument e.
- `try` S_1 `catch` x S_2: execute S_1. If S_1 terminates normally (i.e., without an uncaught exception), the `try` statement also terminates normally. If S_1 raises an exception with value e, the variable $x \in L$ is assigned the value e, and then S_2 is executed.

These are intended to have the standard exception semantics from languages like Java, C#, or OCaml *except* that the catch block merely assigns to x, it does not bind it to a local scope. So, catch does not behave like a let (simplifying the specification of the construct, if not its actual use!). We thus expect:

```
x := 0 ;
{ try
    if x <= 5 then throw 33 else throw 55
  catch x
    print x } ;
while true do {
  x := x - 15 ;
  print x ;
  if x <= 0 then throw (x*2) else skip
}
```

to output "`33 18 3 -12`" and then terminate with an uncaught exception with value `-24`.

> Give the big step operational semantics inference rules (using our new judgment) for the two new statements listed above.

Question 4, Big or small?, *(15 points).* Argue for or against the claim that it would be more natural to describe "WHILE with exceptions" using small-step semantics. You may use "simpler" or "more elegant" instead of "more natural" if you prefer. Do not exceed two paragraphs (one can suffice). Your answer should show an understanding of the differences between big- and small-step operational semantics. If you're not sure where to start, think about situations in which big versus small-step semantics are useful or less useful.

Question 5, Induction, *(35 points).* In the lecture notes, we observed that we can use induction on the structure of expressions to prove that the big- and small-step semantics for `Aexp` obtain

equivalent results. For the syntactic categories in WHILE, we can express this claim formally as:

$$\begin{array}{llllll} \forall a \in \texttt{AExp}. & \forall E.\, \forall n \in \mathbb{Z}. & \langle E, a\rangle \to_a^* n & \Leftrightarrow & \langle E, a\rangle \Downarrow n & (1) \\ \forall P \in \texttt{Bexp}. & \forall E.\, \forall b \in \{\texttt{true}, \texttt{false}\}. & \langle E, P\rangle \to_b^* b & \Leftrightarrow & \langle E, P\rangle \Downarrow b & (2) \\ \forall S \in \texttt{Stmt}. & \forall E, E' \in \texttt{Var} \to \mathbb{Z}. & \langle E, S\rangle \to^* \langle E', \texttt{skip}\rangle & \Leftrightarrow & \langle E, S\rangle \Downarrow E' & (3) \end{array}$$

Prove by induction on the structure of derivations that, if a statement terminates, the big- and small-step semantics for WHILE will obtain equivalent results (equation (3) above). You may assume (1) and (2) have been proven. Show (a) the base case(s), (b) the inductive case for `assign`, and (c) the inductive case for `let` (using the semantics you developed in question (1)). Make sure your proof is sufficiently detailed.

If needed, here are the rules which define $\langle E, S\rangle \to^* \langle E', S'\rangle$ (the reflexive, transitive, closure of $\langle E, S\rangle \to \langle E', S'\rangle$):

$$\frac{}{\langle E, S\rangle \to^* \langle E, S\rangle}\ R \qquad\qquad \frac{\langle E, S_1\rangle \to \langle E', S_2\rangle \quad \langle E', S_2\rangle \to^* \langle E'', S_3\rangle}{\langle E, S_1\rangle \to^* \langle E'', S_3\rangle}\ T$$

If needed, you may assume the following Lemmas hold:

Lemma 1.

Transitivity of $\langle E, S\rangle \to^* \langle E', S'\rangle$

$$\frac{\langle E, S_1\rangle \to^* \langle E', S_2\rangle \quad \langle E', S_2\rangle \to^* \langle E'', S_3\rangle}{\langle E, S_1\rangle \to^* \langle E'', S_3\rangle}$$

Lemma 2.

a) *Small-step assignment congruence of* $\langle E, S\rangle \to^* \langle E', S'\rangle$

$$\frac{\langle E, a\rangle \to_a^* a'}{\langle E, x := a\rangle \to^* \langle E, x := a'\rangle}$$

b) *Small-step sequence congruence of* $\langle E, S\rangle \to^* \langle E', S'\rangle$

$$\frac{\langle E, S_1\rangle \to^* \langle E', S_1'\rangle}{\langle E, S_1; S_2\rangle \to^* \langle E', S_1'; S_2\rangle}$$

c) *Small-step let congruence rule(s) of* $\langle E, S\rangle \to^* \langle E', S'\rangle$ *(applies your answer to Question 1(b))*

Question 6, Bookkeeping, *(1 points).* Indicate in a sentence or two how much time you spent on this homework, how difficult you found it subjectively, and what you found to be the hardest part. Additionally, if you have any feedbacks for the class (e.g., any parts of the course which you didn't understand), feel free to elaborate in this question.

Any non-empty answer will receive full credit.